SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning
NAME: E.V.Mahendra reddy
REGISTER NUMBER: 192425214

COURSE OUTCOME COVERED

CO1: *Analyse reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr. G. A. Senthil
QUESTIONS: 10 **Total Marks: 50**

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyse the convergence of value estimation.

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ε-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ε-Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

Experiment 9: TensorFlow and Keras-Based Neural Network for RL

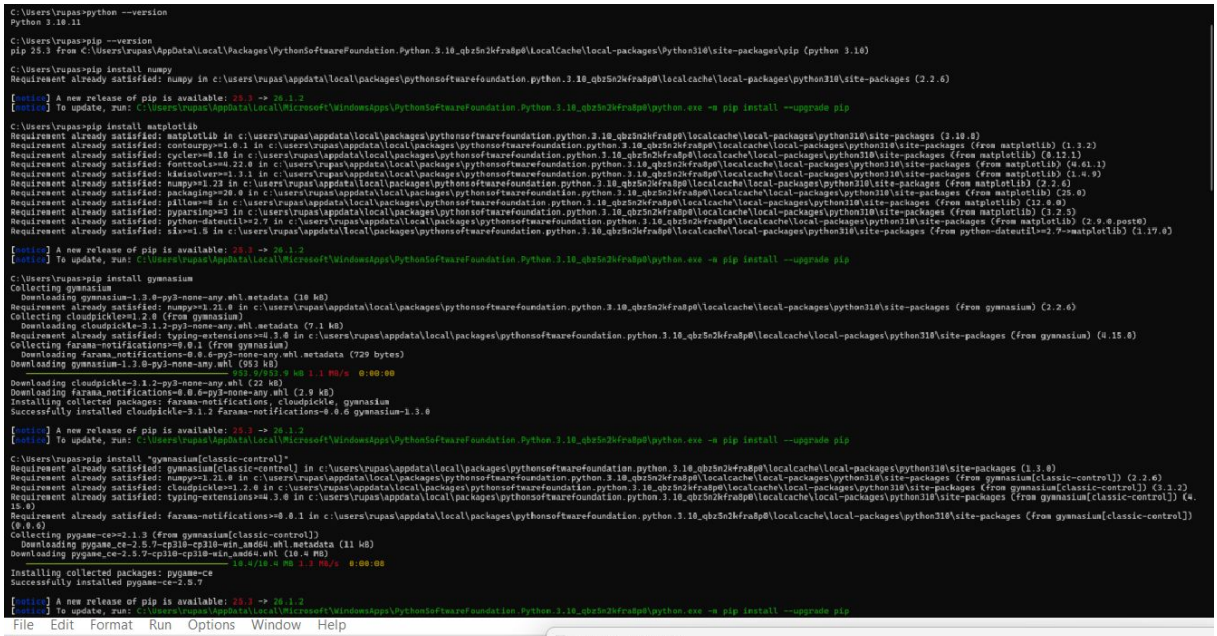
Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments.

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve ~~the CartPole environment and evaluate its learning performance through episode rewards and~~ success rate.

EXPERIMENT-1

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.



File Edit Format Run Options Window Help

```
import numpy
import matplotlib
import gymnasium

print("NumPy Installed Successfully!")
print("Matplotlib Installed Successfully!")
print("Gymnasium Installed Successfully!")
print("Reinforcement Learning Environment Ready!")
```

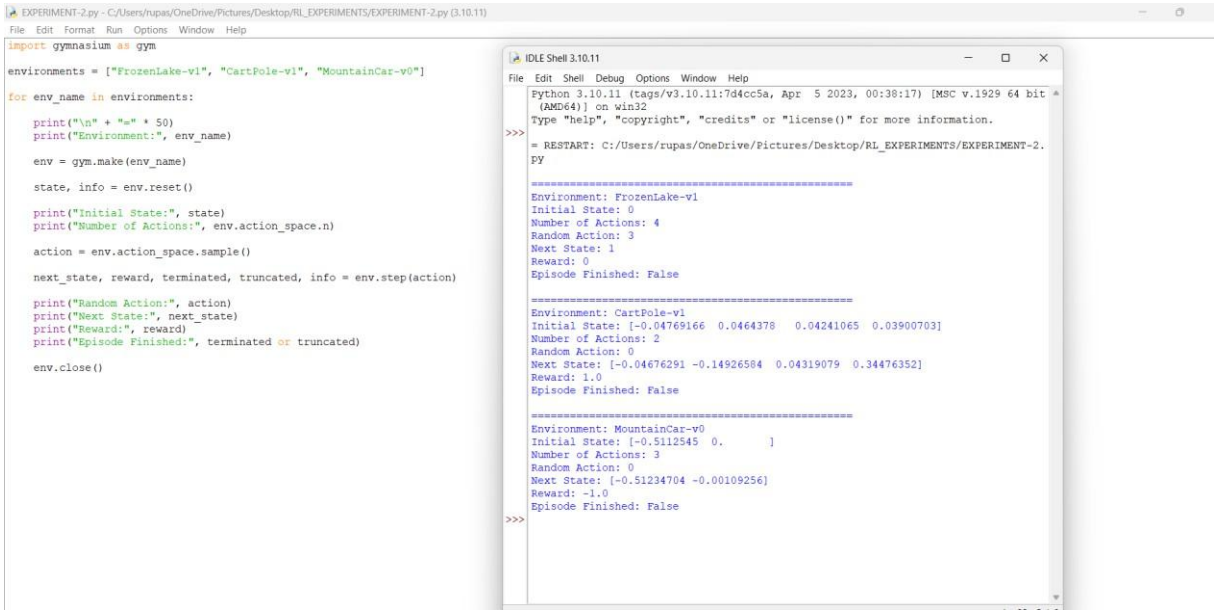
IDLE Shell 3.10.11

File Edit Shell Debug Options Window Help

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more informati
>>>
= RESTART: C:\Users\rupas\OneDrive\Pictures\Desktop\RL_EXPERIMENTS\EX1
py
NumPy Installed Successfully!
Matplotlib Installed Successfully!
Gymnasium Installed Successfully!
Reinforcement Learning Environment Ready!
>>>
```

EXPERIMENT-2

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

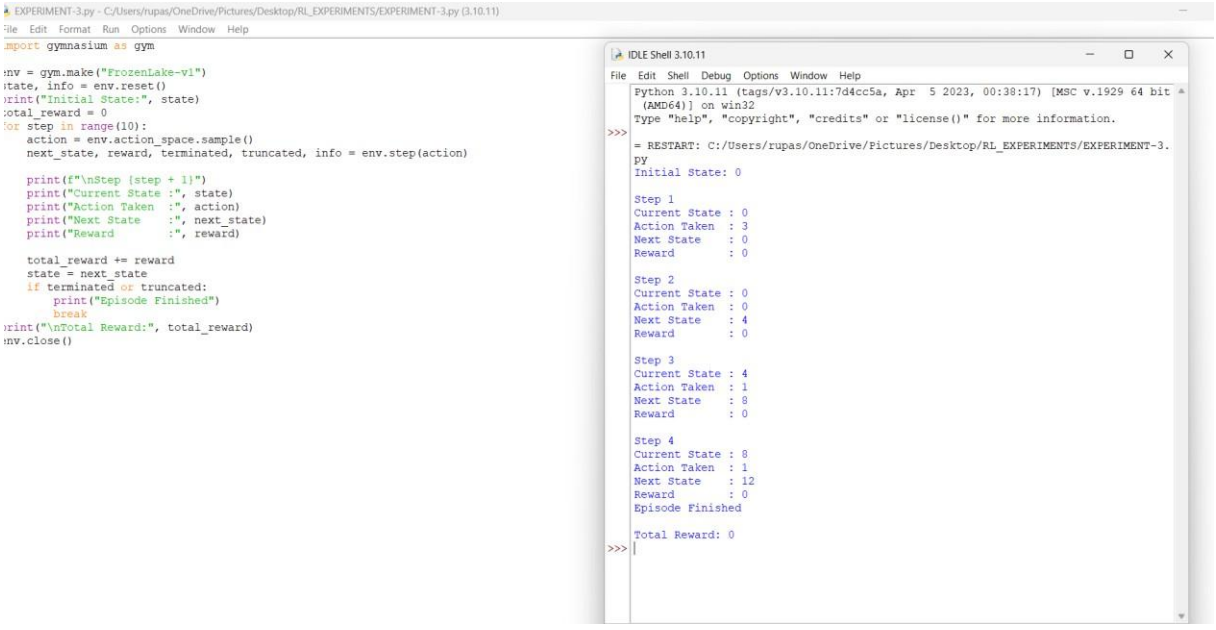


```
EXPERIMENT-2.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-2.py (3.10.11)
File  Edit  Format  Run  Options  Window  Help
import gymnasium as gym

environments = ["FrozenLake-v1", "CartPole-v1", "MountainCar-v0"]

for env_name in environments:

    print("\n" + "=" * 50)
    print("Environment:", env_name)

    env = gym.make(env_name)

    state, info = env.reset()

    print("Initial State:", state)
    print("Number of Actions:", env.action_space.n)

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("Random Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)
    print("Episode Finished:", terminated or truncated)

    env.close()
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-2.
py

==================================================
Environment: FrozenLake-v1
Initial State: 0
Number of Actions: 4
Random Action: 3
Next State: 1
Reward: 0
Episode Finished: False

==================================================
Environment: CartPole-v1
Initial State: [-0.04769166  0.0464378   0.04241065  0.03900703]
Number of Actions: 2
Random Action: 0
Next State: [-0.04676291 -0.14926584  0.04319079  0.34476352]
Reward: 1.0
Episode Finished: False

==================================================
Environment: MountainCar-v0
Initial State: [-0.5112545  0.       ]
Number of Actions: 3
Random Action: 0
Next State: [-0.51234704 -0.00109256]
Reward: -1.0
Episode Finished: False
>>>
```

EXPERIMENT-3

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

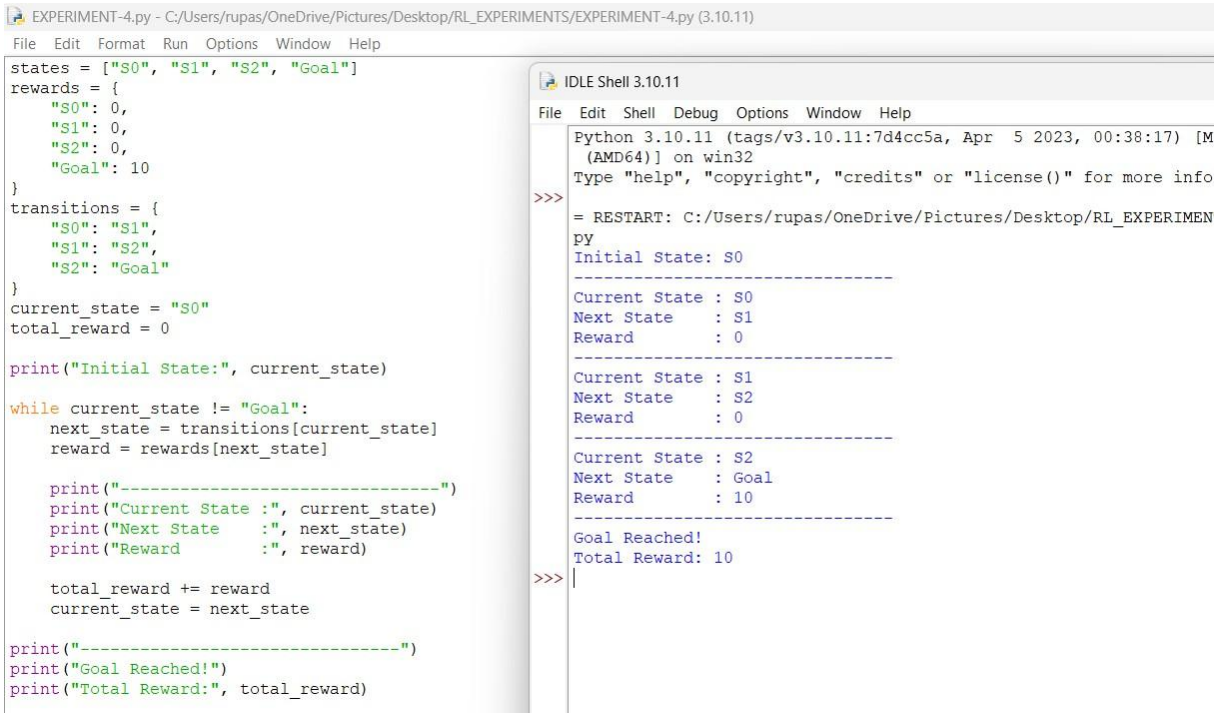


```
EXPERIMENT-3.py - C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-3.py (3.10.11)
File  Edit  Format  Run  Options  Window  Help
import gymnasium as gym

env = gym.make("FrozenLake-v1")
state, info = env.reset()
print("Initial State:", state)
total_reward = 0
for step in range(10):
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)

    print(f"\nStep {step + 1}")
    print("Current State :", state)
    print("Action Taken  :", action)
    print("Next State    :", next_state)
    print("Reward        :", reward)

    total_reward += reward
    state = next_state
    if terminated or truncated:
        print("Episode Finished")
        break
print("\nTotal Reward:", total_reward)
env.close()
```

```
IDLE Shell 3.10.11
File  Edit  Shell  Debug  Options  Window  Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-3.
py
Initial State: 0

Step 1
Current State : 0
Action Taken  : 3
Next State    : 0
Reward        : 0

Step 2
Current State : 0
Action Taken  : 0
Next State    : 4
Reward        : 0

Step 3
Current State : 4
Action Taken  : 1
Next State    : 8
Reward        : 0

Step 4
Current State : 8
Action Taken  : 1
Next State    : 12
Reward        : 0
Episode Finished

Total Reward: 0
>>>
```

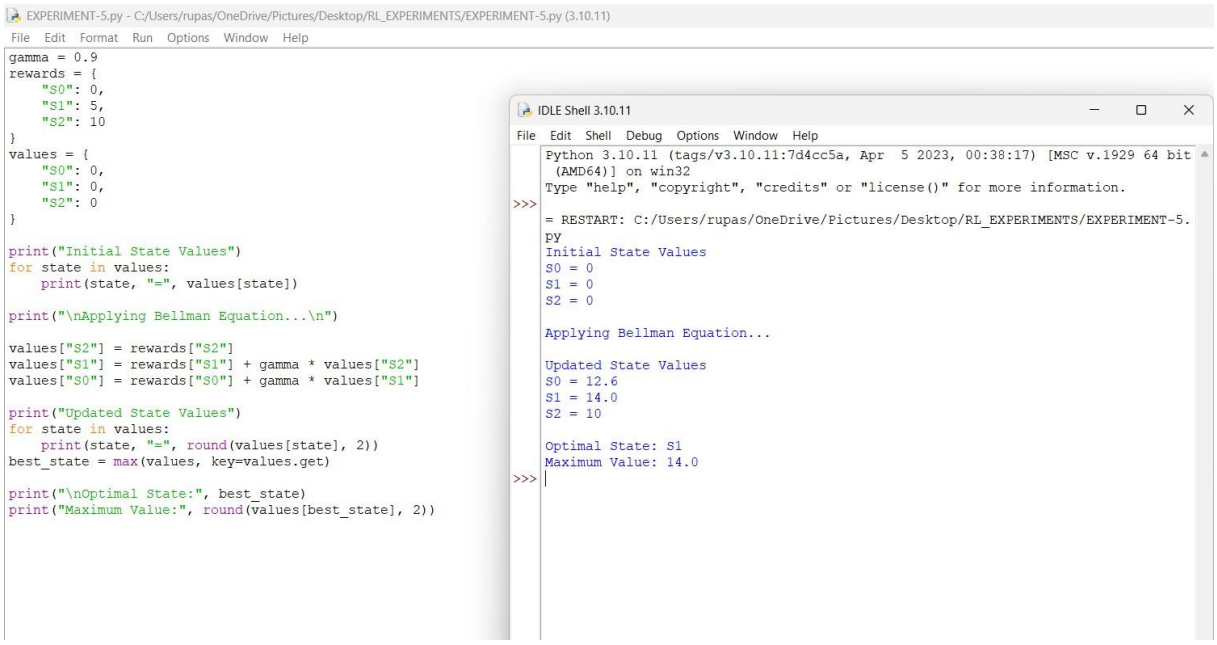
EXPERIMENT-4

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems.



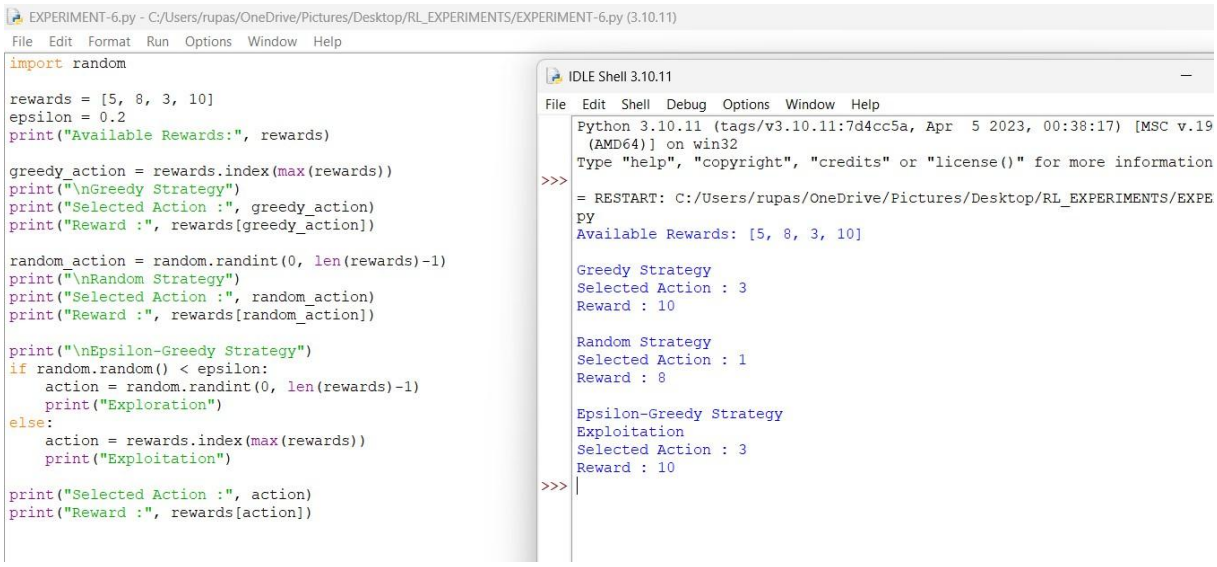
EXPERIMENT-5

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyse the convergence of value estimation.



EXPERIMENT-6

Aim: To implement ε-Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

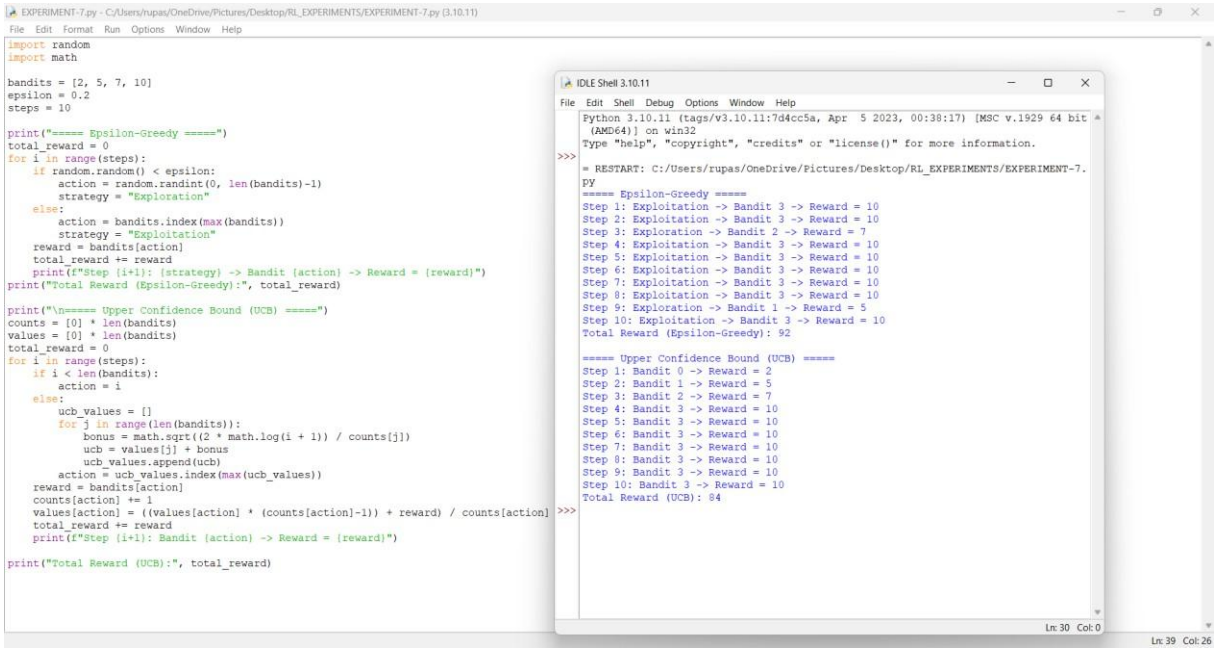


```
import random

rewards = [5, 8, 3, 10]
epsilon = 0.2
print("Available Rewards:", rewards)

greedy_action = rewards.index(max(rewards))
print("\nGreedy Strategy")
print("Selected Action :", greedy_action)
print("Reward :", rewards[greedy_action])

random_action = random.randint(0, len(rewards)-1)
print("\nRandom Strategy")
print("Selected Action :", random_action)
print("Reward :", rewards[random_action])

print("\nEpsilon-Greedy Strategy")
if random.random() < epsilon:
    action = random.randint(0, len(rewards)-1)
    print("Exploration")
else:
    action = rewards.index(max(rewards))
    print("Exploitation")

print("Selected Action :", action)
print("Reward :", rewards[action])
```

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.19
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPE
py
Available Rewards: [5, 8, 3, 10]

Greedy Strategy
Selected Action : 3
Reward : 10

Random Strategy
Selected Action : 1
Reward : 8

Epsilon-Greedy Strategy
Exploitation
Selected Action : 3
Reward : 10
>>>
```

EXPERIMENT-7

Aim: To implement the Multi-Armed Bandit problem using ε-Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods.



```
import random
import math

bandits = [2, 5, 7, 10]
epsilon = 0.2
steps = 10

print("===== Epsilon-Greedy =====")
total_reward = 0
for i in range(steps):
    if random.random() < epsilon:
        action = random.randint(0, len(bandits)-1)
        strategy = "Exploration"
    else:
        action = bandits.index(max(bandits))
        strategy = "Exploitation"
    reward = bandits[action]
    total_reward += reward
    print(f"Step {i+1}: {strategy} -> Bandit {action} -> Reward = {reward}")
print("Total Reward (Epsilon-Greedy):", total_reward)

print("\n===== Upper Confidence Bound (UCB) =====")
counts = [0] * len(bandits)
values = [0] * len(bandits)
total_reward = 0
for i in range(steps):
    if i < len(bandits):
        action = i
    else:
        ucb_values = []
        for j in range(len(bandits)):
            bonus = math.sqrt((2 * math.log(i + 1)) / counts[j])
            ucb = values[j] + bonus
            ucb_values.append(ucb)
        action = ucb_values.index(max(ucb_values))
    reward = bandits[action]
    counts[action] += 1
    values[action] = ((values[action] * (counts[action]-1)) + reward) / counts[action]
    total_reward += reward
    print(f"Step {i+1}: Bandit {action} -> Reward = {reward}")

print("Total Reward (UCB):", total_reward)
```

```
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr  5 2023, 00:38:17) [MSC v.1929 64 bit
 (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/rupas/OneDrive/Pictures/Desktop/RL_EXPERIMENTS/EXPERIMENT-7.
py
===== Epsilon-Greedy =====
Step 1: Exploitation -> Bandit 3 -> Reward = 10
Step 2: Exploitation -> Bandit 3 -> Reward = 10
Step 3: Exploration -> Bandit 2 -> Reward = 7
Step 4: Exploitation -> Bandit 3 -> Reward = 10
Step 5: Exploitation -> Bandit 3 -> Reward = 10
Step 6: Exploitation -> Bandit 3 -> Reward = 10
Step 7: Exploitation -> Bandit 3 -> Reward = 10
Step 8: Exploitation -> Bandit 3 -> Reward = 10
Step 9: Exploration -> Bandit 1 -> Reward = 5
Step 10: Exploitation -> Bandit 3 -> Reward = 10
Total Reward (Epsilon-Greedy): 92

===== Upper Confidence Bound (UCB) =====
Step 1: Bandit 0 -> Reward = 2
Step 2: Bandit 1 -> Reward = 5
Step 3: Bandit 2 -> Reward = 7
Step 4: Bandit 3 -> Reward = 10
Step 5: Bandit 3 -> Reward = 10
Step 6: Bandit 3 -> Reward = 10
Step 7: Bandit 3 -> Reward = 10
Step 8: Bandit 3 -> Reward = 10
Step 9: Bandit 3 -> Reward = 10
Step 10: Bandit 3 -> Reward = 10
Total Reward (UCB): 84
>>>
```

Code of Conduct:

I E.V.Mahendra Reddy(192425214) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Mahendra Reddy

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand